

Resumen 75.44

Resumen 75.44.....	1
Clase 01.....	2
Artful Making.....	2
Clase 02.....	4
Jeff Patton - The game has changed.....	4
Producto.....	4
Product Vision Board (Tablero de visión del producto).....	6
Riesgos.....	7
Riesgo de usabilidad.....	8
Riesgo técnico.....	9
Riesgo de viabilidad.....	9
Ejemplo general.....	9
Entrevistas.....	9
Canvas de experimentos.....	10
Clase 03.....	12
Usability.....	12
Clase 04.....	14
Scrum.....	14
BDD.....	16
Clase 06.....	20
User Story Maps.....	20
Clase 07.....	21
Mejoras.....	21
Métricas.....	23
Retrospectiva.....	25

Clase 01

Artful Making

Es un concepto que intenta explicar lo que los managers de proyectos deberían saber sobre las personas que se dedican al arte. Imitando la forma en la que colaboran los artistas en lugar de el modelo de gestión tradicional los puede llevar a generar propuestas de valor económico. Es decir debemos proponer soluciones flexibles y comunicarnos constantemente para afrontar el dinamismo que pueda tener un desarrollo. Todo debería ser más orgánico.

Entonces el término “Artful” implica que las personas requieren una actitud similar a los artistas y “Making” porque requiere que el trabajo sea llevado a cabo como si se estuviera construyendo un objeto combinando materiales hasta cumplir con un propósito particular, necesita de un hacedor.

Cualquier cosa que requiera construirse completamente nuevo requiere este concepto “Artful Making”. Por ejemplo, algo tan simple como responder exitosamente al movimiento de un competidor.

Esta forma de pensar sostiene que lo que se construya de esta forma será confiable, fácil de entender y capaz de alcanzar niveles de sofisticación que procesos llevados a cabo de forma “científica” no podrán alcanzar.

Sostienen que, en términos de costos, los procesos de arte que son colaborativos son mejores modelos que la mayoría que están basados en reglas que son procesos científicos.

Artful Making difiere de lo que se conoce como Industrial Making en el énfasis que se detalla la planificación, así como objetivos, procesos y productos muy bien especificados. Los principios de Industrial Making son: planificar, producir, crear especificaciones. No se hace nada antes de saber que todo se puede hacer, es decir, se trata de hacer todo bien a la primera, pero esta forma de pensar lleva a disminuir la innovación de los procesos.

Cada forma de planear (Artful Making / Industrial Making) debe hacerse en los contextos correctos, ninguna es exclusiva ni trata de reemplazar a la otra. Ambas formas se complementan e incluso se pueden usar de forma combinada.

Las cuatro cualidades de Artful Making

1. **Release:** Es un método de control que acepta una amplia variación dentro de parametros conocidos. Es lo opuesto a “restraints” o restricciones que es el método de control usado en el proceso industrial.
2. **Collaboration:** Es una cualidad que se exhibe en conversaciones y discusiones. En donde cada parte considera las contribuciones del otro como material de trabajo y no como material de discusión. De esta forma nacen nuevas ideas.
3. **Ensemble:** Es una cualidad que se da por el trabajo en conjunto de varias personas que trabajan para lograr crear algo que ninguno de ellos hubiese podido solo.
4. **Play:** Es la cualidad que se exhibe por la interacción entre los miembros de un grupo.

Clase 02

Jeff Patton - The game has changed

1. Según Jeff Patton, ¿cuál es uno de los principales problemas al manejar ideas para nuevos productos o características?

La mayoría de las ideas que se nos ocurren son malas, por suerte se nos ocurren muchas

2. ¿Cuál de los siguientes conceptos, según Jeff Patton, se centra en los cambios observables en el comportamiento o situación de los usuarios como resultado directo de usar un producto?

Outcomes

3. Alguien dice "tenemos que liberar un MVP". Hay distintos sentidos asignados a MVP ¿cómo podemos saber si se refieren al "smallest successful release"?

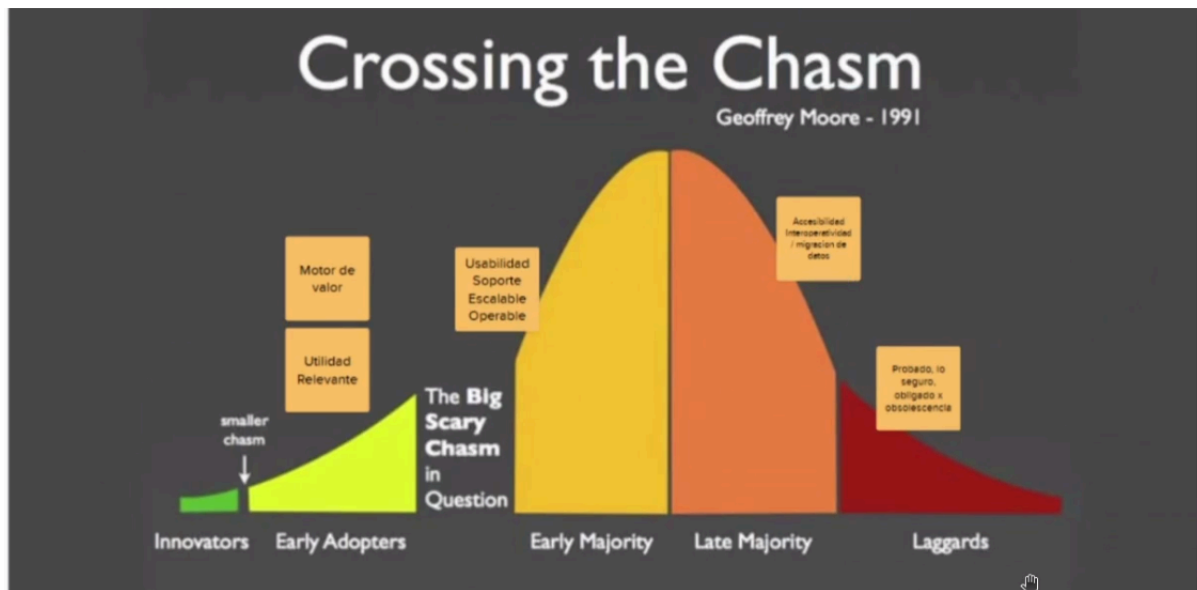
Buscan liberar la versión más pequeña que logre impacto para nuestra organización.

4. Cuando hablamos de aprendizaje validado, ¿cómo lo describirías?

Seleccionar la suposición más riesgosa para nuestro producto y hacer un experimento sobre eso

Producto

Etapas de los usuarios de un producto



- **Los innovadores:** Son un grupo de personas que está interesada en usar un producto que nadie más utiliza. Son las personas que nos pueden dar un primer feedback del producto, pero así como son activas y dispuestas a probar mi producto también son personas que se aburren rápido y empiezan a probar rápidamente otro producto. Es decir es un feedback poco representativo por las características de la personas, no son usuarios ideales a largo plazo.
- **Los que adoptan tempranamente el producto:** Si hay algo que genere valor dentro del producto habrá algo que hará que se sigan usando. A partir de este momento empezarán a sumarse nuevas personas porque ven algo interesante en nuestro producto. Estas personas son las que ven mucho valor en el producto que están dispuestos a usarlo aunque aun sea nuevo y no esté tan pulido.
- **La mayoría temprana:** Es un momento de crecimiento acelerado, cuando el producto se establece y se pule. Este movimiento ocurre hasta que ocupa la cuota de mercado que necesita el valor que ofrece el producto.
- **La mayoría tardia:** Estos son los que usan el producto porque los demás lo usan. Es lo popular y está establecido completamente.
- **Los rezagados:** Estos son los que llegan tarde y usan el producto cuando no les queda de otra más que usarlo por obligación o por última opción.

Ciclo de vida de desarrollo de un producto

1. **Sense:** Se escucha y se aprende. Son personas que están mirando cambios de tecnologías, que cambios tiene la competencia, que problemas están

teniendo los usuarios, etc. Todo con el objetivo de generar ideas y, a partir de esto surgen oportunidades o respuestas para mejorar el producto para los usuarios.

2. **Focus:** Se construye la visión del producto y cual va a ser la estrategia a usar, se definen los resultados u objetivos que esperan obtener de los clientes que usarán la aplicación (el outcome).
3. **Discovery:** En este caso se analizan las ideas que se quieren aplicar, se categorizan para saber si son buenas o malas, es una etapa de diseño o experimentación antes de desarrollar. En este caso se empieza a gestar lo que se va a desarrollar, si estamos en este punto es porque no sabemos si nuestras ideas van a generar el resultado esperado.
4. **Delivery:** Acá se construye el producto.

Los equipos de producto continuamente realizan estas etapas

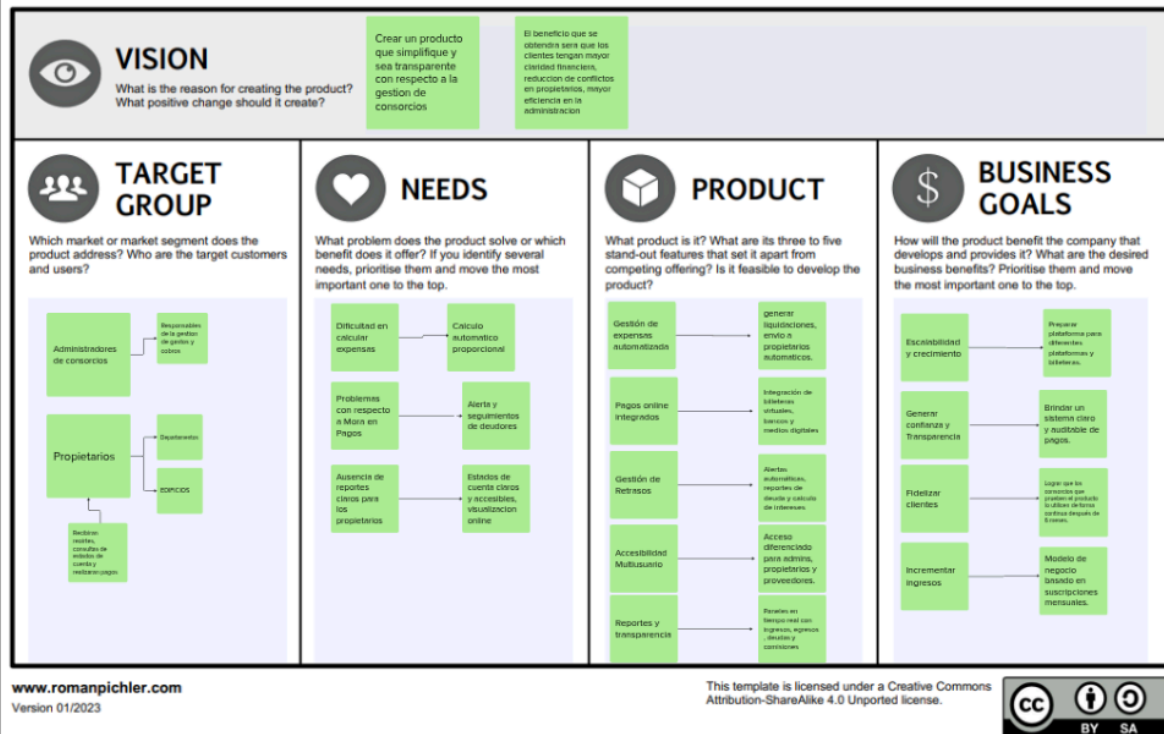
Esta es la clase en donde se vio product vision board, riesgos de los productos y entrevistas.

Product Vision Board (Tablero de visión del producto)

Es una herramienta visual que sirve para **definir y comunicar la visión de un producto**. Ayuda a todo el equipo (negocio, desarrollo, diseño, etc.) entienda **qué problema se resuelve, para quién y por qué**.

Componentes principales

PRODUCT VISION BOARD



1. **Visión:** Qué objetivo general tiene el producto o qué cambio busca generar.
2. **Segmento de clientes:** Quiénes son los usuarios o clientes principales.
3. **Necesidades del cliente:** Qué problemas, frustraciones o deseos se busca resolver.
4. **Producto:** Qué características o funcionalidades lo hacen valioso.
5. **Valor para el negocio:** Qué beneficios obtiene la organización (ingresos, posicionamiento, reducción de costos, etc.).

Para qué sirve:

- Alinear al equipo sobre el propósito del producto.
- Guiar decisiones de diseño y priorización de funcionalidades.
- Evitar desviaciones: mantener el foco en los objetivos.

Riesgos

Qué son:

Son **incertidumbres** o posibles eventos que pueden afectar negativamente el éxito del producto. En gestión de productos se consideran tanto riesgos **técnicos, de negocio, de usuario**, como **de mercado**.

Tipos comunes:

- **Riesgo de valor:** ¿El producto realmente resolverá un problema real del usuario?
- **Riesgo de usabilidad:** ¿El usuario podrá entender y usar el producto fácilmente?
- **Riesgo técnico:** ¿Podemos construirlo con la tecnología disponible?
- **Riesgo de viabilidad:** ¿Es sostenible económicamente o estratégicamente para la empresa?

Para qué sirve identificarlos:

- Priorizar validaciones tempranas (por ejemplo, mediante prototipos o pruebas de usuarios).
- Prevenir pérdidas de tiempo y recursos.
- Ajustar la estrategia del producto antes de invertir demasiado.

Ejemplo de riesgos por tipo

Riesgo de valor

- **Qué es:** El producto podría no generar valor real para el usuario o no resolver su necesidad principal.
- **Ejemplo:** Los usuarios no encuentran útil una app de tareas porque ya usan herramientas similares.
- **Mitigación:** Realizar entrevistas, encuestas o pruebas de concepto con usuarios antes de desarrollar el producto.

Riesgo de usabilidad

- **Qué es:** El usuario podría no entender o no saber usar el producto correctamente.

- **Ejemplo:** Los usuarios abandonan el registro porque la interfaz es confusa o poco intuitiva.
- **Mitigación:** Crear prototipos de baja fidelidad y realizar pruebas de usabilidad antes del lanzamiento.

Riesgo técnico

- **Qué es:** Existen dudas sobre la capacidad tecnológica para construir o mantener el producto.
- **Ejemplo:** La integración con un sistema externo no soporta el volumen de datos esperado.
- **Mitigación:** Hacer pruebas técnicas o prototipos limitados (por ejemplo, pruebas de carga o tests de APIs).

Riesgo de viabilidad

- **Qué es:** El producto podría no ser rentable o sostenible desde el punto de vista económico o estratégico.
- **Ejemplo:** Los costos de infraestructura en la nube superan los ingresos estimados.
- **Mitigación:** Realizar un análisis financiero y técnico temprano; evaluar modelos de negocio alternativos o ajustes de costos.

Ejemplo general

- **Riesgo:** Los usuarios podrían no confiar en una app de salud que recopila datos personales.
- **Tipo:** Riesgo de valor (confianza del usuario).
- **Mitigación:** Agregar explicaciones claras de privacidad, certificados de seguridad y control de datos por parte del usuario.

Entrevistas

Qué son:

Son conversaciones estructuradas con usuarios potenciales o actuales para entender sus necesidades, comportamientos y percepciones sobre el producto.

Tipos de entrevistas:

- **De descubrimiento:** Antes de desarrollar el producto, para entender problemas o hábitos.
- **De validación:** Para comprobar si una idea o prototipo soluciona realmente el problema.

Para qué sirven:

- Obtener información real, no suposiciones, sobre el usuario.
- Validar hipótesis del *Product Vision Board*.
- Detectar oportunidades de mejora o nuevas funcionalidades.

Ejemplo:

- Entrevistar a 10 estudiantes para entender cómo organizan sus tareas antes de diseñar una app de productividad.

Canvas de experimentos

Es una plantilla visual que te ayuda a planificar, ejecutar y analizar experimentos para validar hipótesis sobre tu producto o tus usuarios.

En otras palabras, sirve para probar si tus ideas son correctas antes de invertir tiempo y dinero en desarrollarlas.

En la gestión de productos, puede utilizarse para mitigar los riesgos identificados durante el desarrollo.

Objetivo

Reducir la incertidumbre del producto: comprobar si lo que creés que los usuarios quieren es realmente cierto.

Por eso se usa después de crear hipótesis en herramientas como el Product Vision Board o el Lean Canvas.

¿Por qué es importante?

- Permite experimentar rápido y barato antes de desarrollar una funcionalidad completa.
- Evita invertir en ideas que no generan valor real.
- Ayuda a tomar decisiones basadas en datos, no en intuición.
- Fomenta una cultura de aprendizaje continuo dentro del equipo.

¿Cómo se conectan con las etapas de desarrollo de producto?

Herramienta	Propósito	Etapas
Product Vision Board	Define el propósito del producto y sus usuarios	Inicio
Entrevistas	Descubre necesidades reales de los usuarios	Descubrimiento
Canvas de Experimentos	Valida hipótesis sobre esas necesidades o soluciones	Validación temprana
Gestión de riesgos	Evalúa y mitiga los riesgos identificados en las hipótesis	Planificación / ejecución

Clase 03

Usability

Es una metodología de investigación de UX:

*“In a **usability-testing** session, a researcher (called a “facilitator” or a “moderator”) asks a participant to perform tasks, usually using one or more specific user interfaces. While the participant completes each task, the researcher observes the participant’s behavior and listens for feedback.”*

Normalmente los objetivos de las pruebas de usabilidad son:

- Identificar problemas en el diseño del producto o servicio
- Descubrir oportunidades de mejora
- Aprender sobre el comportamiento y las preferencias de los usuarios objetivos

Elementos principales en las pruebas de usabilidad

Los facilitadores, las tareas y los participantes. Los facilitadores administran las tareas de los participantes, mientras que los participantes realizan las tareas los facilitadores observan el comportamiento de los participantes y toman nota del feedback que les puedan dar. Los facilitadores también pueden realizar preguntas para buscar más información del participante.

Tipos de pruebas de usabilidad

Las pruebas de usabilidad pueden ser cualitativas o cuantitativas:

*“**Qualitative usability testing** focuses on collecting insights, findings, and anecdotes about how people use the product or service. Qualitative usability testing is best for discovering problems in the user experience. This form of usability testing is more common than quantitative usability testing.”*

*“**Quantitative usability testing** focuses on collecting metrics that describe the user experience. Two of the metrics most commonly collected in quantitative usability testing are*

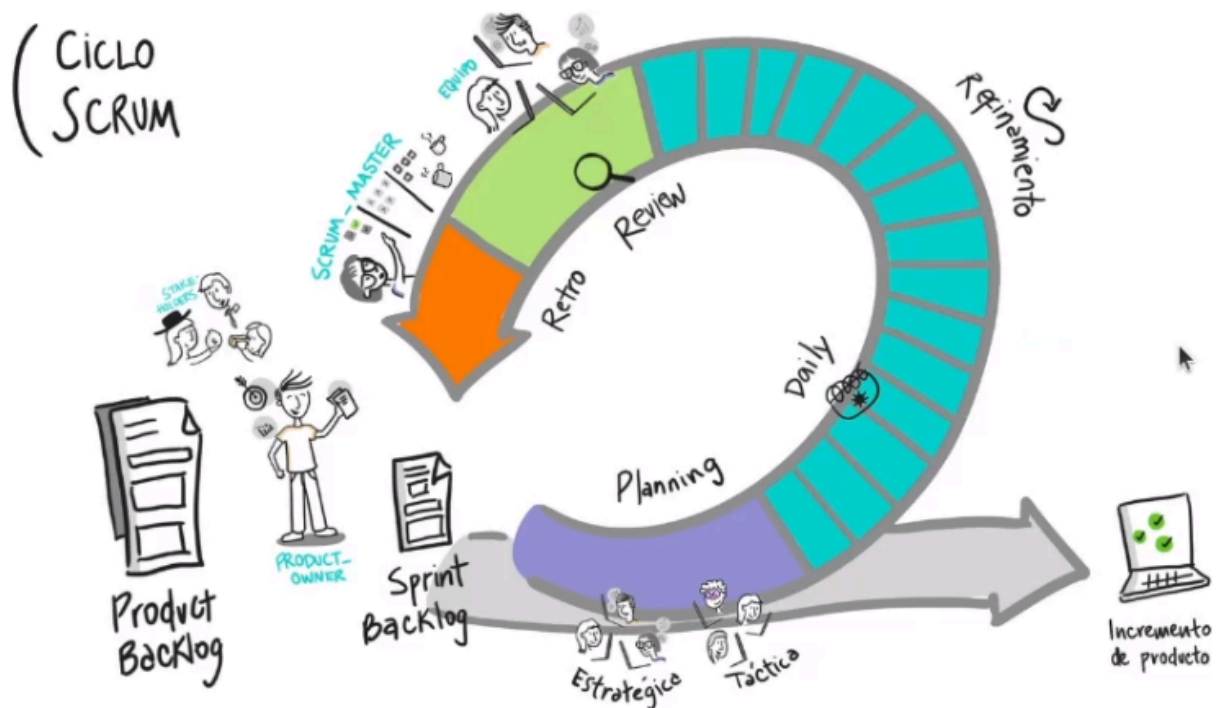
task success and time on task. Quantitative usability testing is best for collecting benchmarks.”

Clase 04

Esta clase consiste en desarrollo de producto y queremos saber qué es lo que necesitamos para hacerlo o cómo lo vamos a hacer, para ser más exacto sería como vamos a verificar que lo que se va a desarrollar va a cumplir con las necesidades del usuario, cómo nos aseguramos que cuando el desarrollo haya terminado cumpla con todos los requerimientos que se establecieron en caso de tenerlos (criterios de aceptación). Esta es una actividad que ocurre posterior al descubrimiento del producto. En este punto tendremos un prototipo y varias tareas que sabemos que los usuarios requieren. Las tareas o títulos que tenemos de la etapa del descubrimiento necesitan ser detalladas para poder comenzar a realizar el producto.

En este caso vamos a estudiar dos técnicas para detallar los requerimientos del usuario, son enfoques para la definición y entrega de producto, Scrum y BDD (Son marcos para procesos de desarrollos)

Scrum



- **Product backlog:** Son cosas o ideas que se descubrieron en la etapa de descubrimiento. Basicamente contiene todo lo que el producto pudiera tener, no significa que lo vamos a desarrollar sino que contiene una lista de mejoras que el equipo puede elegir hacer para mejorar el producto.

- **Incremento de producto:** Es una de las mejoras implementadas por el equipo de desarrollo que tomaron del backlog y la concretaron. Basicamente una vez que un item del backlog se completa segun sus criterios de aceptación entonces nace un incremento de producto que es una parte del producto que se está desarrollando, es un escalon que nos permite acercarnos al objetivo del mismo.
- **Sprint backlog:** Es un plan que dice que objetivos de corto plazo necesitamos para acercarnos al objetivo de largo plazo, basicamente en un plan tactico para estar un paso más cerca del objetivo de producto. Contiene una serie de ítems del backlog que son las que el equipo va a trabajar para cumplir con el sprint correspondiente. Los ítems se eligen de acuerdo a los riesgos y las prioridades de cada uno.

Estructura iterativa de scrum (Eventos / Reuniones dentro del proceso)

- **Planning:** En estas reuniones se planifica que items se van a tomar del backlog, se miran los criterios de aceptacion para saber si se pueden hacer, se arma un plan tactico para llevar el sprint a cabo.
- **Daily:** Son reuniones para hablar sobre cómo va el sprint, si hay que cambiar algo, o no dentro del plan, es decir, son reuniones cortas.
- **Refinamiento:** Es una actividad cuyo proposito es preparar los elementos del backlog que no están en el sprint backlog, es decir no están dentro del sprint actual, lo que se refina es el trabajo de los siguientes sprints. No tienen un momento específico en el tiempo para llevarlo a cabo, se da cuando se necesite y las veces que sea necesario, lo define el equipo, porque el objetivo es preparar el trabajo a futuro (por ejemplo ven una tarea dentro del backlog muy larga y deciden partirla en dos o varias partes, pero es algo que se desarrollara igual a futuro es ir pensando las cosas que están por hacerse).
- **Review:** Se revisa el avance del sprint, si se lograron los objetivos o no, que se logró dentro del mismo, se muestra lo que se desarrolló para que los stakeholders lo usen.
- **Retrospectiva:** Cuida que ocurra exitosamente todo lo que se necesita para que el equipo siga funcionando bien, y es una etapa para encontrar qué problemas tuvo el equipo y como se puede mejorar de cara a la siguiente iteracion. En este caso lo que buscan es entender cómo fue el proceso de desarrollo y no los items que se desarrollaron.

Responsables (roles) dentro del ciclo de scrum

Entre todos conforman el equipo de Scrum

- **Desarrolladores:** Personas técnicas que saben hacer el producto.
- **Product Owner:** Persona que sabe alinear el trabajo de los desarrolladores para cumplir con el objetivo. Es la persona responsable de lograr que el trabajo que realiza el equipo sirva para cumplir con el objetivo del producto.
- **Scrum Master:** Es un facilitador. Ayuda al equipo a tener su mejor versión, los aconseja y acompaña durante el proceso de scrum, es el responsable de ese proceso, se encarga de que se cumplan las prácticas del framework de scrum. No tiene responsabilidades sobre el producto. Es responsable del proceso de scrum que es el que se usa para desarrollar el producto.

BDD

- Es una aproximación ágil a la construcción del software
- Define los comportamientos que debe tener el sistema para servir a los objetivos del equipo, la definición del comportamiento son definidos colaborativamente entre desarrolladores y negocio
- Las especificaciones son documentadas en un lenguaje compartido
- Se pueden usar para construir tests y automatizarlos

Entonces en resumen, BDD busca guiar el desarrollo incremental mediante la promoción de conversaciones sobre el comportamiento esperado del sistema y la producción de especificaciones ejecutables (cucumber, visto en algo 3 era un framework para trabajar con BDD).

Actividades dentro de BDD

- **Discovery**
Se genera un entendimiento compartido mediante colaboración y conversaciones estructuradas.
- **Formulation**
Los ejemplos del comportamiento del sistema son documentados como escenarios.
- **Automation**
Los escenarios son automatizados para obtener feedback mecanizado del comportamiento del sistema.

Para resolver la ambigüedad entre el ***discovery de producto y el discovery de bdd*** usamos las siguientes definiciones.

Product Discovery

Se refiere a descubrir qué producto hacer

BDD Discovery

Se refiere a definir en detalle cada característica del producto

Flujo del proceso de BDD



Esta parte tiene que ver con desarrollar la historias, hacer las reglas y los ejemplos para cada una.

Example mapping (Mapa de ejemplos)

Es una técnica usada en el descubrimiento en BDD

Notacion:

- En amarillo se colocan las historias de usuario
- En azul se colocan las reglas de negocio
- En verde se colocan los ejemplos

Historia / Feature	Regla de Negocio	Ejemplos / Casos
--------------------	------------------	------------------

Carga del carrito de compras	Permitir agregar bebidas alcohólicas a mayores de edad	Mayores de edad - El usuario tiene 18 años o más. - Abre la lista de categorías. - Selecciona la categoría “<i>Bebidas Alcohólicas</i>”. - Agrega un producto de la categoría. → El carrito contiene el producto seleccionado. Menores de edad - El usuario tiene menos de 18 años. - Abre la lista de categorías. - Selecciona la categoría “<i>Bebidas Alcohólicas</i>”. - Muestra error indicando que es una categoría restringida para mayores. → No se permite agregar el producto.
Carga del carrito de compras	El carrito actualiza la cantidad de productos	Agregado de un producto - El carrito está vacío. - El usuario selecciona una categoría. - Agrega un producto. → Se agrega el producto al carrito. → La imagen del carrito muestra un “1”. Agregado de varias unidades de un producto - El carrito contiene un ítem. - Selecciona la misma categoría. - Agrega el mismo producto. → El carrito contiene 3 unidades del producto agregado. → La imagen del carrito muestra un “3”.

Post rojo: “¿Sería mejor que directamente no le mostremos la categoría?”

En este caso de ejemplo permitir bebidas alcoholicas hay una regla que es de decision y es logico pensar en dos casos distintos. El de menores de edad es un caso en donde explicitamos que sucede con el sistema cuando la regla no se cumple, por eso la pregunta en rojo, que dice que pasa si no dejamos pedir a los menores de edad en lugar de cubrir ese comportamiento con un mensaje explicativo.

Clase 06

User Story Maps

Es una técnica utilizada en la concepción de un producto: permite esbozar un nuevo producto o una nueva característica para un producto existente,

El resultado es un diagrama en que todas las historias de usuario están ordenadas en grupos funcionales. Esto ayuda a no perder de vista el panorama general, a la vez que proporciona todos los detalles de la aplicación en su conjunto.

Un User Story Mapping facilita el descubrimiento del producto y la priorización del trabajo de construcción. Esto se consigue poniendo las actividades y tareas del usuario en un mapa que sirve para mantenerlas en contexto.

El User Story Mapping muestra cómo cada historia de usuario individual se inserta en el conjunto de la aplicación. Esto permite detectar vacíos y realizar una priorización consistente.

Es una forma de organizar las ideas del producto distinta al backlog, porque el backlog carece de contexto y no tiene un orden en particular, solamente conoce de prioridades pero no está relacionado con cómo los usuarios van a utilizar esas funcionalidades, cual es el valor que les aportan y cómo se ordenan esas funcionalidades de forma secuencial si es posible.

Clase 07

Inicio de contenido segundo parcial

Mejoras y Métricas

Mejoras

Prácticas que nos ayudan a mejorar:

- Reduciendo desperdicios (TOYOTA)
- Metricas
- Retrospectiva

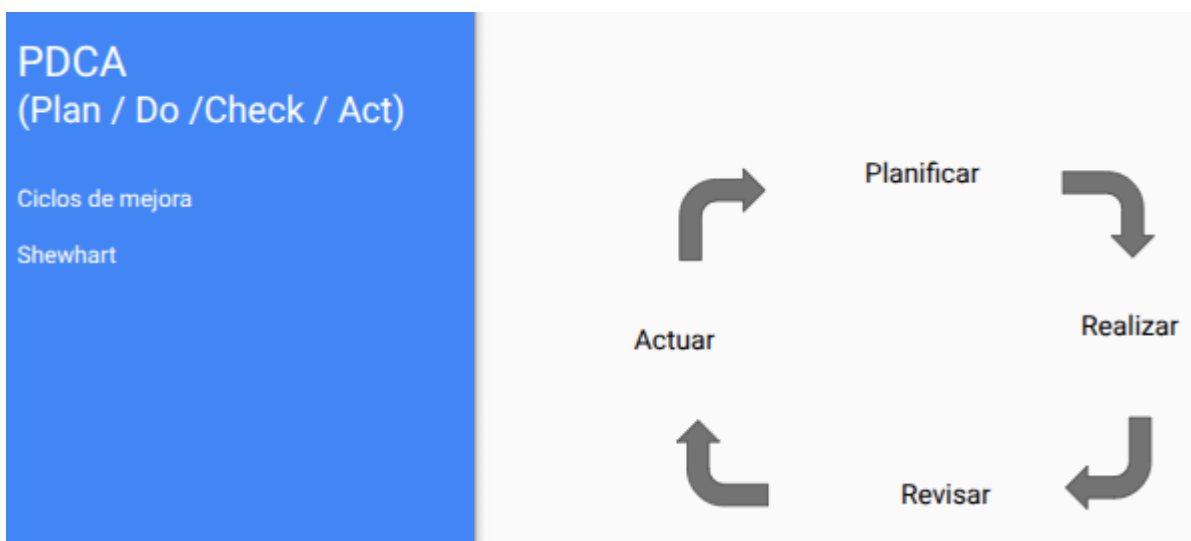
Razones para mejorar

- Porque todo empeora de por sí (ejemplo, entropía). Los procesos se degradan de manera progresiva.
- No todo nos sale natural. Como no todo se nos da bien, entonces si queremos buscamos hacer un esfuerzo para mejorar y cambiar.
- Nuestros valores están por encima de nuestra práctica actual.
- Como el mundo es complejo, perdemos eficacia si no nos adaptamos. Necesitamos adaptarnos a los cambios.
- Siempre hay algo por aprender.

¿Cómo mejoramos en la industria en general?

Los procesos de software que son exitosos, en su mayoría son iterativos. Porque sabemos que debemos aprender, y mejorar durante el proceso de desarrollo.

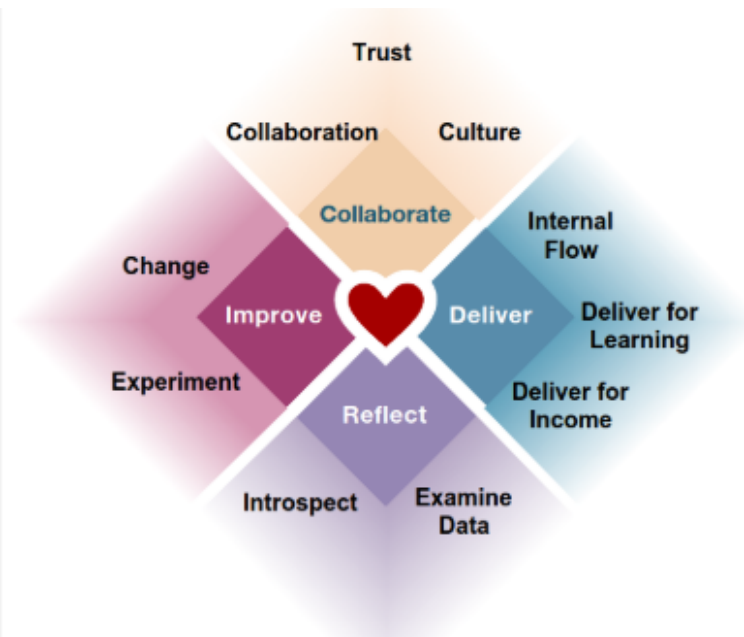
Ciclos de mejora



Heart of Agile

Ciclos de mejora

Alistair Cockburn



Observación: la retrospectiva no produce productos, sino produce cambios en la forma en la que hacemos productos.

Objetos de mejora

Se refiere a los fines o ambiciones que se establecen para un período determinado, con el fin de lograr un progreso incremental y mejorar continuamente en un área específica de la actividad.

- Procesos y prácticas (proceso es el flujo de las cosas y práctica es finalmente como se hacen)
- Equipo
- Herramientas
- Producto
- Dirección (¿A dónde vamos?)
- Organización
- Consciencia

Mejoras concretas en desarrollo de software

Lean Thinking (Toyota)

- Mejora continua (kaizen)

改善 = cambio + bueno

Kaizen quiere decir cambio bueno

- Mejora radical (kaikaku)

改革 = cambio + revolucionario

Kaizen quiere decir cambio radical, disruptivo

Ejemplos:

Kaizen: eliminar el requisito de revisión cruzada cuando hacemos pair programming.

Kaikaku: implementar continuous deployment (todo commit va a producción)

Mejora como reducción de desperdicios (TOYOTA)

En este caso lo que quiere decir es que hacemos mejora reduciendo los desperdicios.

1. Trabajo en progreso (Es decir podemos reducir el trabajo en progreso, mejor tener menos cosas terminadas que muchas abiertas sin terminar)
2. Funcionalidades extras (Hacer lo mínimo de lo mínimo, agregar cosas extras innecesarias es un gran desperdicio)
3. Reaprendizaje (Cosas que aprendemos, olvidamos y tenemos que volver a aprender)
4. Traspasos (Hand off, pasar la información de un lado a otro, si pasa por muchos puntos es más probable que se haga algo que no es lo que se pidió)
5. Cambio de contexto (Cambiar de una tarea a otra sin hacerlo bien, nuestra conciencia tiene que cambiar a la siguiente tarea y olvidarse de la anterior, pero mientras más tareas más riesgo de equivocarse)
6. Demoras (Bloqueos por trabajos que no se pueden realizar, que tienen dependencias o que no se están haciendo en paralelo)
7. Defectos (Cosas que se hacen mal y que después hay que arreglarlas nuevamente)

Mejoras concretas en desarrollo de software

- Ajustar criterios de aceptación (trabajar en las condiciones con la cual vamos a evaluar si el ítem está cumpliendo con todo lo que se espera de él)
- Realizar acciones preventivas (ej. quitar warnings)
- Adoptar nuevas prácticas/procesos
- Aprender técnicas
- Pulir nuestra técnica
- Cambiar secuencias de acciones
- Crear redundancias (más personas saben hacer deploy)
- Unificar flujos (los requerimientos entran por un único lugar)
- Hacer visibles los problemas
- Profundizar nuestro conocimiento/estudiar
- Reemplazar herramientas
- ¿Otras?

Métricas

Las métricas no deben ser medidas de éxito o fracaso, **sino de realidad (verdad)**. Solamente si no son medidas de éxito o fracaso podemos esperar que las personas no

actúen buscando soluciones fáciles. Básicamente te dice que las métricas miden la realidad y que no debemos enfocarnos inicialmente en que son éxito o fracaso. (Tobias Mayer)

¿Para qué usamos las métricas?

- Para caracterizar un recorte de la realidad (clave, sobre todo está relacionado a la definición anterior). No es todo lo que piensan las personas, pero la métrica nos da un primer pantallazo de la realidad.
- Para mejorar el proceso, la práctica, el servicio, etc.
- Para cuantificar y comparar con parámetros esperados.
- Para controlar, monitorear: evaluamos nuestro proceso y decidimos acciones
- Para analizar la calidad del proceso o producto

Ejemplos de métricas y objetivos

- Métrica: Cantidad de ítems del backlog terminados sobre planificados
Objetivo asociado: Desplegar versión de producto que pueda ser usada en una fecha
- Métrica: Cantidad de defectos hallados post entrega por iteración
Objetivo asociado: Lograr 0 defectos por iteración
- Métrica: % de cobertura de pruebas sobre condicionales (Alto es mejor):
Objetivo asociado: Llegar a 80% de cobertura sobre condicionales.
- NPS (Net Promoter Score) (Alto es mejor):
¿Cuán probable es que recomiende X?

Conceptos importantes

- Métrica: La definición formal de una variable que vamos a medir.
- Medición: El acto de tomar valores para una métrica.
- Indicador: La representación visual de una métrica.

Ejemplos de buenas métricas

- Satisfacción de clientes / empleados*
- Defectos en la Scrum Review / o en prod
- % pruebas automatizadas / total
- % de pruebas especificadas sobre pruebas totales
- Duración del build / pruebas automatizadas*
- Demora del despliegue de un commit*
- % de avance para un release

Elementos de una métrica

- Nombre
- Escala
- Valor más positivo (Es definir hacia qué lado va la escala, o sea que valor es muy bueno y qué valor es muy malo. Ejem cuan satisfecha estas del 1 al 5 con 5 mas positivo 1 menos positivo)

- Forma de medición (Ponernos de acuerdo en cómo vamos a medir)
- Estadístico de agregación (la función que convierte los valores medidos en métrica)

¿Cómo diseñamos una métrica?

Nos planteamos un objetivo, en base a ese objetivo nos hacemos preguntas y para cada pregunta definimos métricas que nos ayuden a responderlas.

Ejemplo:

Objetivo 1: Entregar Producto orgánico Periódicamente

Pregunta 1.1: ¿Entregamos producto a nuestro usuario en cada sprint?

Métrica 1.1.1: Equipo entrega producto frecuentemente: Sí/No

Métrica 1.1.2: Cantidad de incrementos del sprint entregados

Pregunta 1.2: ¿Los incrementos de producto generan valor al usuario?

Métrica 1.2.1: Equipo entrega valor : Sí/No

Métrica 1.2.2: Nivel de satisfacción de usuarios (1-5)

Métrica 1.2.2: Cantidad de incrementos del sprint entregados (reusada)

Retrospectiva

(OJO: Mencionado en scrum anteriormente)

Propósito

Analizar una situación, detectar causas y establecer compromisos de mejora

Estructura

- Ingreso: conectarse con la mejora (tenemos que tener presente que es lo que queremos mejorar)
- Recolección de datos (cuantitativamente o cualitativamente)
- Priorización y análisis
- Propuestas de mejora > Compromisos
- Salida: conectarse con el compromiso

¿Cuándo puede ser necesario plantear una retrospectiva?

- Al final de cada iteración (Scrum)
- Para analizar incidentes productivos (Por ejemplo se juntan personas de todos los equipos para determinar que sucedió y como se puede evitar)
- Cuando hay un conflicto particular
- Luego de un hito importante
- Cuando sea necesaria

Tecnicas aplicadas durante las etapas de la retrospectiva

1. Recolectar datos
 - a. ¿Preguntar qué hicimos bien/mal? - ¿Que hay que Cambiar/Mantener?
 - b. Línea de tiempo (datos de las cosas que van ocurriendo en el tiempo para saber que fue ocurriendo en el transcurso. es un poco más para procesos más largos tal vez)
 - c. Errores/Aciertos/Agradecimientos/Ideas
 - d. Métricas
2. Análisis
 - a. 5 Por qué (Es una heurística para encontrar la causa raíz de un problema, dar varias razones de por que las cosas tuvieron un resultado en particular. Preguntar por que 5 veces de forma recursiva como los filosofos, por que a todo basicamente)
 - b. Agrupamiento temático
3. Compromisos
 - a. Seleccionar 1
 - b. Sacrificio rotativo (una persona hace algo que no le gusta hacer y va cambiando todos los dias)
 - c. Alguien siempre hace progreso (Ejemplo una persona que programe siempre todos los dias, así los que no lo hacen siempre puedan acudir a esa persona)

Clase 08

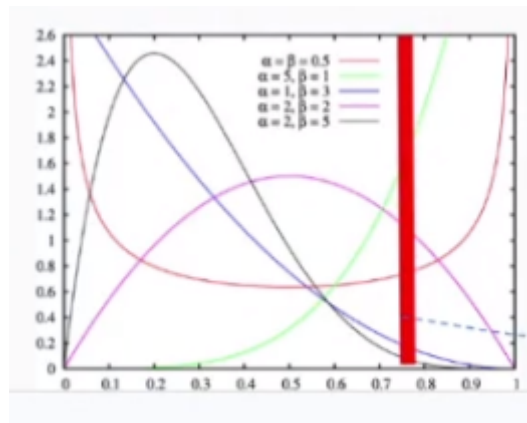
Estimaciones, seguimientos y control con indicadores

Estimación (Pronóstico) vs Presupuestos (Compromisos)

Estimar es proponer un valor que creemos probable para una variable. Una estimación razonable en general es un rango. Por ejemplo: creo que puede llevar entre 3 y 5 días

Un **presupuesto** o compromiso establece un acuerdo entre partes. Por ejemplo: Por \$ x puedo hacer Y.

Todo presupuesto o compromiso debe agregar a las variables estimadas un componente en funcion de los riesgos. Por ejemplo: Si puede llevar entre 3 y 5 días, pero no hay margen de demoras podemos comprometernos a que esté listo en 7 días.



Estimaciones

¿Para qué?

- Para saber que esperar que pase si “no se hacer nada”, si todo va como normalmente debería ir.
- Para planificar y asignar recursos en etapas tempranas, mitigar riesgos y dimensionar tiempos y costos.
- Ayudan a priorizar tareas, evitar sobrecargas y ajustar expectativas realistas.
- Para saber si hacer o no hacer.

¿Para quién?

- Principalmente para las personas involucradas fuertemente (Equipo, Stakeholders, Gestores Internos)

En desarrollo de software consideramos costos de iteración y riesgo. Equivocarnos significa que podemos perder tiempo y dinero.

Tipos de estimaciones:

- Predictiva: Para situaciones bien conocidas.
- Adaptativo: Para situaciones de alta incertidumbre. En desarrollo de software esto es lo comun porque tenemos mucha incertidumbre y tratamos de hacer iteraciones justamente para adaptarnos a ese conocimiento que adquirimos a medida que avanzamos.
- Estadístico: Para situaciones con distribuciones conocidas.

¿Qué estimamos?

- Duracion: ¿Para cuando?
- Costo: ¿Cuánto cuesta?
- Esfuerzo: ¿Cuántas horas o gente?
- Objeto: ¿Que? (proyecto, producto, hito, requerimiento, tarea)

Teoría de estimación

- Toda estimación implica probabilidad de acertar (si fuera segura, no sería estimar). No son predicciones sino pronósticos
- Un buen estimador es insesgando, es decir debe tener la misma probabilidad de equivocarse por exceso o defecto. Un ejemplo de agregar un sesgo, es recibir un bono por cumplir con tiempos de estimaciones, desde ese momento trataremos de estimar con mayor probabilidad de éxito porque un bono depende de acertar, estamos atando un bono a cumplimiento de hitos
- En desarrollo tendemos a subestimar la complejidad, es decir “no habia previsto X”. Podemos medir la complejidad en funcion de lo que conocemos o nos imaginamos, es difícil estimar lo que no se conoce.
- Es una habilidad que se aprende.

Otra definicion de estimación es:

- Descomponer el problema en partes, estimar/contar las partes. Pero luego tenemos un problema y es como volvemos a un valor único de esas partes (Componemos)
- Componer: sumar, multiplicar por cantidades, integrar determinística o estadísticamente para obtener el todo

Metodos de estimación

- Delphi:

Es una técnica de estimación que recopila opiniones de expertos a través de rondas iterativas de cuestionarios anónimos para llegar a un consenso. Este proceso incluye la retroalimentación controlada, donde las respuestas de cada ronda se resumen y se comparten para que los expertos puedan reconsiderar sus juicios en función de los del grupo. Su objetivo es reducir la variabilidad y la incertidumbre para obtener una estimación fiable sobre temas complejos o la predicción de eventos futuros. Hay una variación en donde cada persona define un mínimo, un valor más probable y otro máximo (luego se componen estadísticamente, no se suman directamente). También se suelen utilizar como puntuaciones valores que sean discretos y que se

encuentren dentro de un rango, para evitar que las estimaciones sean muy precisas, lo cual suele requerir mucho conocimiento y suelen ser estimaciones mentirosas en la práctica.

- **Metodo NoEstimate:**

Es el resultado de una tendencia que se fue dando en desarrollo de software al acercarnos más a los procesos de flujo continuo en donde es importante que el tamaño de las tareas sean más chicas y de forma similar (que tengan el mismo tamaño). Si todas son relativamente iguales no hace falta estimarlas todas, solo tenemos que saber cuánto tardamos en la unidad que usemos para cada una tarea.

- **Planning Poker:**

Planning Poker es una técnica de estimación ágil donde un equipo utiliza un juego de cartas para estimar de forma colaborativa el esfuerzo de una tarea o historia de usuario. El proceso implica que cada miembro estime de forma privada y anónima, revele sus estimaciones simultáneamente y luego discutan las diferencias para llegar a un consenso, haciendo que el esfuerzo de estimar sea más rápido, divertido y menos sesgado, por ejemplo, evitando que la estimación de una persona con mayor conocimiento afecte al resto.

Presupuestos

¿Para qué?

- Para conocer lo que quiero que pase así tomaré acciones para lograrlo.
- Para planificar y asignar recursos a niveles organizacionales, gestión de riesgos de cartera de inversiones.
- Funciona como baseline de comparación que permite tomar acciones para el equipo.
- Para colaborar con otros equipos.

¿Para quién?

- Para la dirección de la organización.
- Equipo y stakeholders.
- Otros equipos.

Seguimiento: Indicadores de gestion y calidad

¿Cómo hacemos para hacer un seguimiento? Necesitamos metricas e indicadores

- Una métrica da un valor a una variable.
- Un indicador muestra ese valor visualmente.
- Una medición es recolectar el valor de una variable.
- Durante el seguimiento medimos y comparamos (Ejemplo: para determinar el avance)

Indicadores posibles dentro del desarrollo de Software

Planteados por los alumnos durante la clase

- Reseñas de usuarios
- Generación de ganancias
- Evolución en la cantidad de usuarios
- Uso de funcionalidades
- Porcentaje de completitud del sprint
- Cumplimiento de tareas

Planteado por el profesor durante la clase

- Producto
 - Satisfacción de usuarios
 - Tasa de uso por característica
 - Cantidad de usuarios
 - Volumen de operaciones
 - Ganancias
- Proceso
 - Avance hacia objetivo
 - Lead time (DORA)
 - Frecuencia (DORA)
 - Porcentaje de fallos (DORA)
 - Tiempo de recuperacion (DORA)
- Equipo
 - Satisfacción
 - Valoración de las prácticas / proceso
 - Rotación de integrantes
- Cliente
 - Satisfacción de cliente
 - Duracion de contrato

Segun el manifiesto agil, el software funcionando es la medida principal de progreso

Indicadores de calidad

- Cantidad de defectos por version en producción
- Presencia / cobertura de pruebas automatizadas
- Despliegue automatizado
- Tiempo de recupero ante fallas
- Nivel de adopción de prácticas (ej. TDD o CI)
- Cantidad de tipos de pruebas distintas

Indicadores para el futuro

- **Niveles de adopcion de TDD e integración continua:** Si sabemos que estas son técnicas que nos ayudan a prevenir errores en producción entonces podemos evitar errores en el futuro midiendolas

- **Tiempo de ciclo de revisión de código:** Un tiempo de ciclo corto indica que el código se revisa y se hace merge rápidamente, lo cual es una señal positiva para la velocidad de entrega futura
- **Estabilidad de los requisitos:** Un índice de estabilidad de requisitos bajo sugiere que los requisitos probablemente cambiarán con frecuencia, lo que puede generar retrabajo y demoras futuras
- **Tamaño del Pull Request (PR) y tiempo de recogida:** PR más grandes o tiempos de recogida más largos pueden anticipar un aumento en el tiempo de ciclo general, indicando posibles ralentizaciones futuras
- **Cobertura de pruebas:** Un porcentaje alto de cobertura de pruebas puede ser un indicador adelantado de la calidad y estabilidad del código en el futuro
- **Tasa de retrabajo:** Una tasa alta de retrabajo puede indicar que están ocurriendo problemas de calidad en el código, lo que podría causar problemas futuros si no se abordan
- **Alcance:** La cantidad de alcance que se agrega a un proyecto después de que comienza puede ser un indicador adelantado de riesgos futuros y posibles fallas del proyecto

Técnica	Época	Enfoque	Precisión	Esfuerzo
Análoga	Tradicional	Histórico	Baja	Bajo
Paramétrica	Tradicional	Estadístico	Media-Alta	Medio
PERT (3 puntos)	1950s	Probabilístico	Alta	Alto
Delphi	1950s	Consenso expertos	Media	Alto
Monte Carlo	1940s-90s	Simulación	Muy Alta	Muy Alto
Story Points	2000s	Relativo	Media	Medio
#NoEstimates	2010s	Empírico	Variable	Bajo

Clase 09

Kanban

Es un método de gestión de proyectos visual que utiliza un tablero para organizar y optimizar el flujo de trabajo, ayudando a los equipos a visualizar sus tareas, limitar el trabajo en curso y mejorar continuamente su eficiencia. La palabra japonesa significa "tarjeta visual" o "señalización", lo que refleja su uso de tarjetas (físicas o digitales) que representan tareas y se mueven por columnas que indican diferentes etapas del proceso, como "por hacer", "en progreso" y "terminado". Es comparable con Scrum por ejemplo

Prácticas principales de la metodología

1. Visualizar el flujo de trabajo:

Tenemos que poder representar de manera visible el proceso y también cómo las cosas en las que estamos trabajando fluyen en etapas.

2. Limitar el trabajo en curso (Work In Progress):

En el flujo de trabajo, un equipo dado no puede tener más de una cierta cantidad (parametrizada) de piezas de trabajo (que podrían ser historias de usuarios o tareas) en un estadio dado, por ejemplo: tener un máximo de 3 tareas, con 5 personas entonces las personas van a tener que compartir tareas porque solo se puede trabajar en 3 actividades al mismo tiempo. La razón para reducir el trabajo en curso es priorizar que se terminen cosas más que que se empiecen.

Cualquier columna es work in progress, no necesariamente la columna de in progress.

3. Gestionar el flujo de trabajo:

Cuidamos la organización del trabajo para que las cosas avancen. Por ejemplo:

- Dos personas desarrollando con un único tester, si los desarrolladores son muy rápidos y entregan en simultáneo el tester no va a poder trabajar en las dos cosas al mismo tiempo, va a estar ocupado en una cosa, entonces, que podríamos hacer, podríamos hacer que un desarrollador cambie de rol de desarrollo a tester para probar el desarrollo de su compañero. Esto es gestionar el flujo de trabajo, el propósito de la gestión es reducir el trabajo del tester para que las tareas no se acumulen
- Mirar las piezas de trabajo que hay en cada etapa del proceso para detectar cuellos de botellas, encontrar por qué sucede, si siempre es así, etc.

4. Explicitar las políticas:

Los criterios para decidir cosas están explicitados claramente. Por ejemplo:

- Limitar el trabajo en progreso de cada una de las columnas dentro del proceso de gestión y que esté visible y claro.
- Todas las personas pueden tomar cualquiera de los roles dentro del equipo.
- O las personas con ciertas características del equipo solo harán trabajos relacionados con ellos, es decir sin cambio de roles.
- Las tareas se asignan a pedido, es decir, el primero que se libera elige el siguiente trabajo a realizar
- Se asignan las tareas al azar apenas una persona se libera.

5. Implementa bucles de retroalimentación:

El trabajo siempre se va colocando más adelante dentro del proceso o flujo de trabajo, nunca hacia atrás, lo que sí puede retroceder es el feedback o la retroalimentación. Por ejemplo:

- Una historia testeada que se encuentra con errores no puede pasar a un estado de “despliegue” pero tampoco puede volver hacia atrás. entonces iría a un estado siguiente llamado “re trabajo” y al mismo tiempo comunicar a la persona que lo trabajó sobre lo que sucedió, que falló, etc (dar información confirmando o diciendo que ocurrió).
- El proyecto viene lento, estamos avanzando pero a medida que avanzamos nos cuesta más llegar a producción. De alguna manera teniendo el proceso organizado en etapas podemos ir midiendo cuánto tardamos en cada paso para tener este tipo de información.

La idea es organizar el proceso para que el feedback sea lo más temprano posible.

6. Mejorar colaborativamente con experimentos:

Es mejorar como equipo con el uso de experimentos para confirmar o rechazar las hipótesis que nos hacemos para cumplir estas mejoras. Por ejemplo: Si pasar a producción es mucho trabajo y complejo capaz convenga colaborar más entre infraestructura y desarrollo para facilitar este trabajo (esto es un experimento, lo que haríamos para probar si ayuda a mejorar nuestro flujo de trabajo)

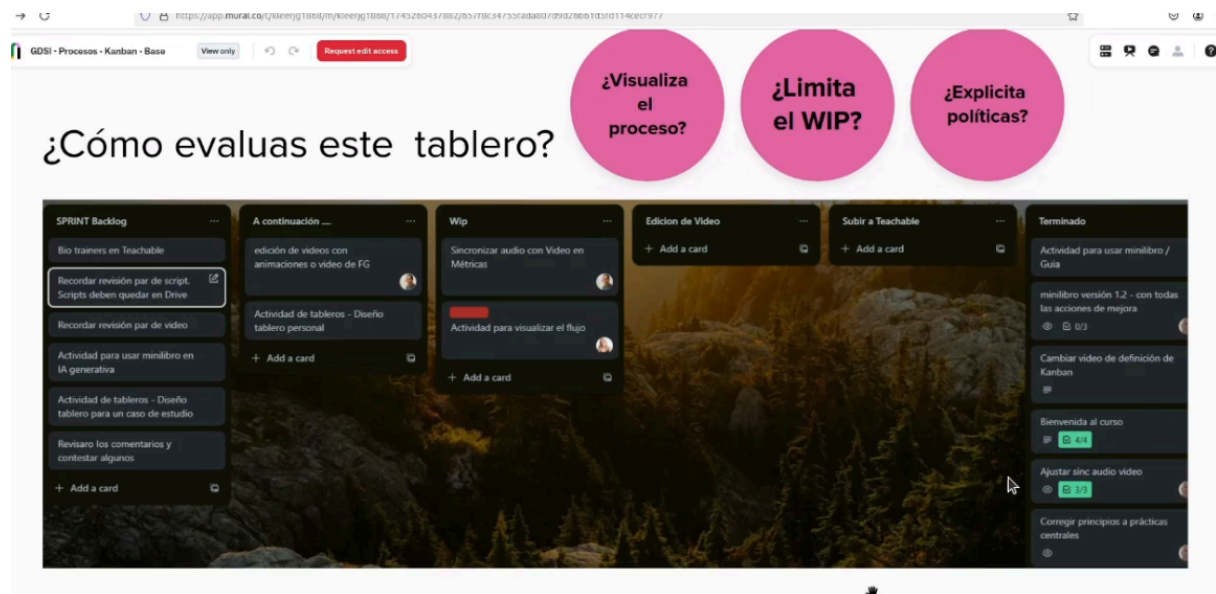
Principios de gestión de cambios

¿Cómo se gestiona el cambio en Kanban? ¿Cuál es la idea?

- Comenzar visualizando lo que estamos haciendo ahora y a partir de esto podemos empezar a mejorar
- Acordar la búsqueda de la mejora a través del cambio evolutivo, es decir, tiene que haber un acuerdo de que la mejora es algo que vamos a perseguir a medida que vamos haciendo pequeños cambios que nos permitan ir a mejorar.
- Fomentar actos de liderazgo a todos los niveles. Liderazgo que podría ser simplemente tomar iniciativa o liderazgo por estar a cargo de algo.

Principios de Orientación al Servicio

- Comprender y enfocarse en cumplir las necesidades y expectativas del cliente, es decir, tratar de empatizar y mirar que es lo que está pasando, que representa un pedido del cliente dentro de su contexto de trabajo. No es hacer por hacer, es hacer cosas que estén claras y que cumplan con lo que quiere el cliente
- Gestionar el trabajo; dejar que los trabajadores se auto organicen en torno a él
- Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados entregados. Esto quiere decir que hay que estar controlando todos los servicios que se están desarrollando para saber cómo se conectan, como se pueden afectar entre sí, etc.



Clase 10

Desarrollo de Proyecto

El desarrollo de un proyecto se orienta a entregar algo específico dentro de un tiempo determinado.

Características principales

- Tiene inicio y fin definidos.
- Objetivo fijo: entregar un alcance (“scope”) acordado.
- Éxito = cumplir plazo, presupuesto y alcance.
- Se trabaja con un cliente específico.
- Se suele planificar con metodologías tradicionales (Waterfall) o híbridas.
- Una vez entregado, el equipo se disuelve o pasa a otro proyecto.

Ejemplos:

- Implementar Salesforce para un cliente.
- Construir una app corporativa para uso interno.
- Migrar un sistema legacy en 6 meses.

Desarrollo de Producto

El desarrollo de un producto se orienta a crear y mejorar algo que se utiliza de manera continua y evoluciona según el mercado/usuarios.

Características principales

- No tiene fin definido: es continuo.
- Objetivo variable: mejorar, escalar, agregar valor y mantener al producto vivo.
- Éxito = satisfacción de usuarios, adopción, métricas de negocio.
- El “cliente” es el usuario final (no uno solo: muchos).
- Se trabaja normalmente con metodologías ágiles (Scrum, Kanban).
- El equipo es estable y cuida el producto en el largo plazo.

Ejemplos

- Gmail como producto en constante evolución.
- Un SaaS, como Salesforce.
- Una app para clientes que se actualiza permanentemente.

Resumen de desarrollo de proyecto vs desarrollo de producto

Aspecto	Desarrollo de Proyecto	Desarrollo de Producto
Duración	Temporal, fin definido	Continua, sin fin fijo

Enfoque	Entregar un alcance	Mejorar valor continuamente
Métrica de éxito	Plazo, costo, alcance	Uso, satisfacción, crecimiento
Usuarios	Un cliente	Muchos usuarios
Equipo	Cambia por proyecto	Equipo estable de producto
Cambio de requisitos	Costoso o restringido	Esperado y continuo
Responsabilidad	Se entrega y termina	Se mantiene y evoluciona

Gestion de equipos y personas

Tipos de personas en la gestión

Los sistemas se orientan hacia una de estas dos formas y las personas tienen que adaptarse en la medida de sus posibilidades a este tipo de gestion.

- Personas X (no autonomas y necesitan ser supervisadas)

Las personas se comportan de forma correcta para el trabajo que están haciendo, solo si se les aclara con mucha claridad el trabajo que tienen que hacer y se controla con bastante rigurosidad y detalle que lo estén cumpliendo.

- Personas Y (autonomas y no necesitan ser supervisadas)

Las personas son autónomas, automotivadas y buscan cumplir sus objetivos y obtener sus resultados. Lo único que tenemos que hacer es darles un contexto y un soporte apropiado para que puedan llevar a cabo sus actividades. Basta con un objetivo claro y los elementos necesarios para cumplirlo.

“Construye los proyectos alrededor de individuos motivados. Provee el entorno y soporte que ellos necesitan, y confía en que haran el trabajo”. Manifiesto para el desarrollo ágil.

Gestión de Equipos

“Las mejores arquitecturas, requerimientos, y diseños emergen de equipos auto organizados”. Manifiesto para el desarrollo ágil.

- ¿Cuántas personas debería tener un equipo de desarrollo?

Segun el manifiesto ágil, 7 +/- 2 personas, si el proyecto es muy grande entonces se usarán varios equipos, cada uno con esos tamaños y que cada uno va a tener cierto nivel de autonomía y puede ser necesario que se tengan que comunicar entre ellos.

- Ley de Conway:

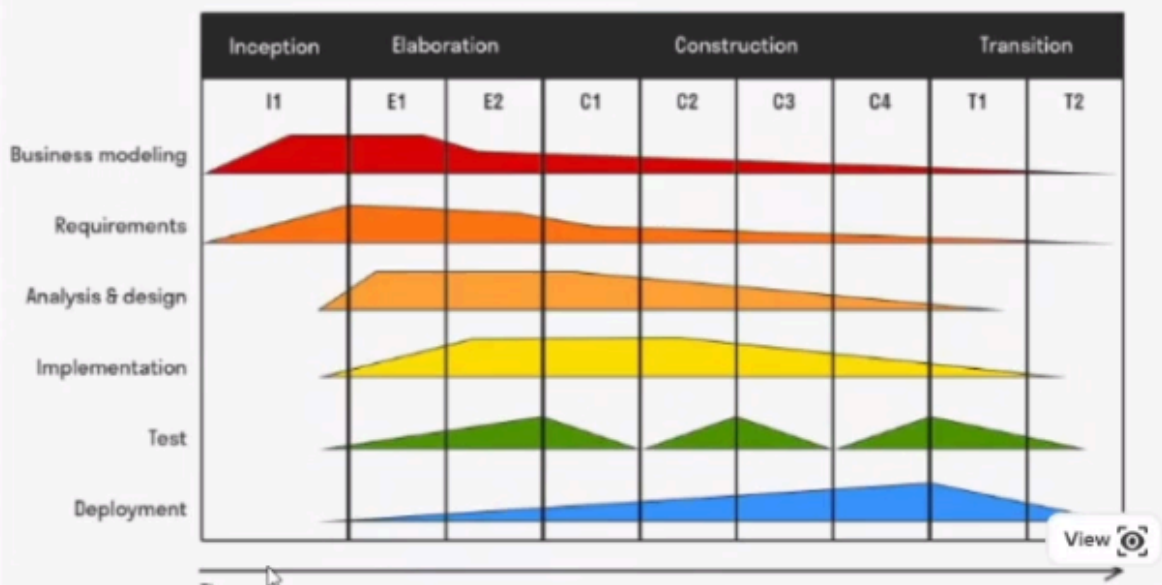
La estructura del equipo tiene que estar en relación con la arquitectura del producto. Y la solución debería tener componentes más o menos independientes.

Existen dos formas de conformar los equipos:

- *Component teams*, que son equipos que se organizan por componentes en los cuales se están trabajando (especialidades básicamente). Por ejemplo, tener un equipo para trabajar en android, otro para ios, otro para web, otro para el backend. Esto puede llegar a ser un problema cuando llega un nuevo requerimiento del negocio porque los equipos pueden llegar a tener que desarrollar lo mismo en cada componente y se generan dependencias entre ellos, si un equipo se retrasa, se retrasa al resto y puede generar fricciones entre ellos, entonces parecerían equipos separados pero terminan comportándose como uno solo. En este caso surge la necesidad de un rol que identifique estas dependencias, que sepa qué equipo va a tener que involucrarse.
- *Feature teams*, en este caso son equipos que se dividen por funcionalidades, las personas no son tan especialistas en cada tema sino tienen un conocimiento más general. Puedes tener un feature que deba tocar ios, android, web y backend y dentro del mismo equipo estén las personas necesarias para llevarlo a cabo, y ellos determinan cómo se divide cada tarea. Cada persona puede saber más que el otro de un tema pero el equipo en general puede cumplir con toda la feature sin depender de otro.

¿Cuánto tiempo tarda un equipo en ser un equipo? ¿Cuándo deja de ser un grupo de personas para transformarse en equipo?

Una aproximación serían 5 ciclos o sprints lo que se necesita para que se conozcan (no es un valor matemático es solo algo que se fue observando en el tiempo)



Las personas en los equipos de proyecto son transitorios o temporales, como se ve en el gráfico, hay ciertos roles necesarios para ciertas etapas y otras en donde no se utilizan, entonces las personas se van re asignando para seguir utilizando sus habilidades.

¿Cómo se conforman los equipos?

Proyecto:

- La selección de personas es por necesidades de conocimiento y son variables en el tiempo
- La organización de los equipos suelen ser jerárquicas, hay reportes desde los que están abajo hacia arriba
- Los equipos son transitorios o temporales porque los proyectos son de duración finita y suelen hacerse en cascada, una vez que una parte del equipo termina su trabajo rotan a otro proyecto

Producto:

- Las personas se seleccionan a partir de un diseño de alto nivel, y se trata de que tengan habilidades generales.
- La organización es horizontal porque las personas dentro del equipo empiezan a tomar decisiones sobre el producto sin tanta necesidad de especialistas, no hay jerarquías.
- Los equipos suelen ser persistentes en el tiempo

Proceso:

- La selección de las personas viene después de la definición del proceso y en función de eso se convocan a las personas
- Los procesos suelen ser jerárquicos
- Los equipos suelen ser persistentes en el tiempo

Comunicación

- El desarrollo de software es un juego de comunicación e invención
- Hacer software es como una conversación
- Hay que distinguir claramente entre pedidos y ofertas cuando comunicamos
- La transparencia y visibilidad son clave para mantener expectativas razonables

Comunicación no violenta

¿Cómo expresamos las cosas? Podemos observar sin evaluar, para evitar conflictos

Formas que mezclan observación con evaluación	Ejemplos	Observación sin evaluación
Uso del verbo <i>ser</i> , sin asumir responsabilidad de la interpretación realizada.	¡Sos impuntual!	Es la 3ra vez en esta semana que llegas 10 minutos después de la hora acordada.
Uso de verbos con connotaciones evaluativas.	Estás bloqueando el paso.	No queda mucho lugar para pasar por allí, donde estás parado ahora.
Hacer generalizaciones en lugar de citar ejemplos específicos.	Todos los de Marketing no entienden lo que hacemos.	En las dos últimas reuniones con Marketing, me encontré explicando de tres formas diferentes lo que hacemos. Así y todo, no estoy seguro de haber sido claro.
Hacer afirmaciones usando <i>siempre</i> , <i>nunca</i> , <i>rara vez</i> , <i>a menudo</i> .	Mario siempre está ausente al inicio de las reuniones.	Las veces que nos reunimos este año, Mario se incorporó a las reuniones 15 minutos después.
Dar por sentado que las interpretaciones que hacemos son las únicas posibles.	No vino porque no le importa lo que hacemos.	Me pregunto por qué no habrá venido. A veces pienso que, tal vez, este tema no le interese tanto como a nosotros.

Liderazgo

Es la capacidad de tener iniciativa, no quiere decir ser el jefe sino ayudar a avanzar, cambiar o aprender.

Son formales o informales, porque la organización define ese liderazgo, informales porque surgen naturalmente por las características de la persona.

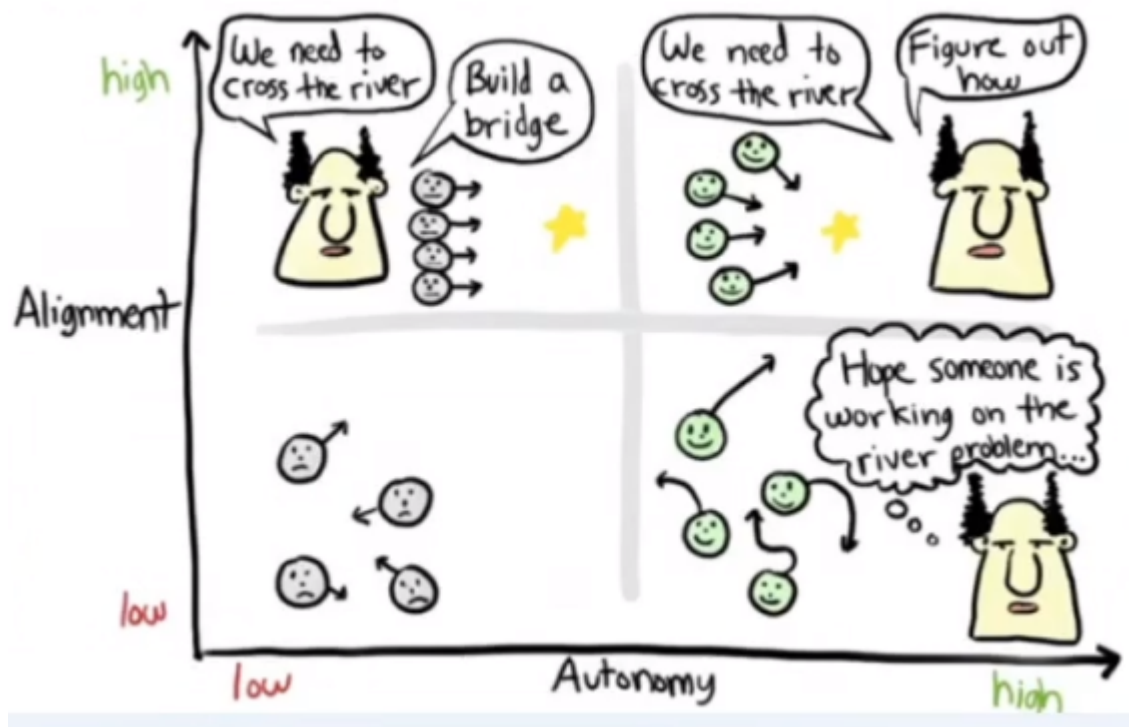
En Scrum:

- El equipo lidera en la tarea y la organización
- El facilitador (scrum master) lidera en la mejora
- El product owner lidera en la dirección del producto

Estilos de liderazgo:

- El visionario: "Vengan conmigo"
- El ejemplar: "Seamos excelentes"
- El democrático: "Podemos encontrar el camino"
- El mentor: "Puedo ayudarlos a ser tan buenos como pueden ser"
- El coercitivo: "Hagan lo que yo diga"
- El integrador: "Las personas primero"

Control autonomía y autoorganizacion



Combinaciones de control y autonomía del equipo:

1. **Baja alineación y baja autonomía:** El equipo no tiene dirección, se encuentra perdido por la falta de liderazgo.
2. **Alta alineación y baja autonomía (Tradicional):** El líder da instrucciones claras al equipo de que es lo que tienen que hacer pero no opinan del proceso solo hacen las tareas porque el líder ya definió como lograr sus objetivos.

3. **Alta alineación y alta autonomía (Agil):** El líder da instrucciones claras al equipo de lo que se necesita realizar y colaboran para encontrar la solución al problema porque el líder les da esa libertad.
4. **Baja alineación y alta autonomía:** Falta supervisión sobre el trabajo de las personas en el equipo e instrucciones claras pero tienen mucha libertad para trabajar, esto puede hacer que no se llegue a cumplir con los objetivos necesarios.

Equipos autoorganizados y multifuncionales (equipos Ágiles)

- La responsabilidad es solidaria (todos tienen responsabilidad por el todo: producto, procesos, tareas, etc)
- Mejora la gestión del trabajo porque el equipo se conoce a sí mismo y a la tarea mejor que nadie
- Aumenta la motivación porque tienen mayor autonomía
- Tienen menos bloqueos porque tienen mayor autonomía
- Tienen menos bloqueos porque tienen menos dependencias externas.
- Pueden organizar mejor el trabajo porque tienen todas las habilidades usualmente necesarias y las personas están dispuestas a hacer las tareas que hagan falta.